

Telemetry / 遥测

8 translated articles / 8 篇译文

Contents / 目录

Telemetry: Recording and Sending Real-Time Data / 遥测：记录和发送实时数据	3
New for 2025: Robot Telemetry with Annotations (Java Only) / 2025 年新增：使用注解实现机器人遥测（仅 Java）	5
Recording Faults with Persistent Alerts / 使用持久警报记录故障	14
Robot Telemetry with Sendable / 使用 Sendable 实现机器人遥测	17
On-Robot Telemetry Recording Into Data Logs / 将机器人端遥测数据记录到数据日志中	19
Downloading & Processing Data Logs / 下载和处理数据日志	23
Writing Your Own Sendable Classes / 编写自己的 Sendable 类	29
Third-Party Telemetry Libraries / 第三方遥测库	31

Telemetry: Recording and Sending Real-Time Data / 遥测：记录和发送实时数据

Recording and viewing [telemetry](#) data is a crucial part of the engineering process - accurate telemetry data helps you tune your robot to perform optimally, and is indispensable for debugging your robot when it fails to perform as expected.

记录和查看[遥测](#)数据是工程流程中的关键环节。准确的遥测数据有助于调校机器人，使其发挥最佳性能；当机器人未按预期工作时，这些数据也是调试不可或缺的依据。

By default, no telemetry data is recorded (saved) on the robot. However, recording data on the robot can provide benefits over recording on a dashboard, namely that more data can be recorded (there are no bandwidth limitations), and all the recorded data can be very accurately timestamped. WPILib has integrated support for on-robot recording of telemetry data via the **DataLogManager** and **DataLog** classes and provides a tool for downloading data log files and converting them to CSV. The Java library also provides a convenient [annotation](#) to autogenerate telemetry logging code based on your project.

默认情况下，机器人不会记录（保存）任何遥测数据。不过，与在仪表板上记录相比，在机器人上记录数据有一些优势：可以记录更多数据（不受带宽限制），且所有记录的数据都可以带有非常精确的时间戳。WPILib 通过 **DataLogManager** 和 **DataLog** 类集成了机器人端遥测数据记录支持，还提供下载数据日志文件并将其转换为 CSV 的工具。Java 库也提供方便的[注解](#)，可根据项目自动生成遥测日志代码。

Note / 说明

In addition to on-robot recording of telemetry data, teams can record their telemetry data on their driver station computer with [Shuffleboard recordings](#).

除了在机器人上记录遥测数据，队伍还可以通过 [Shuffleboard 录制功能](#)，在 Driver Station 计算机上记录遥测数据。

Adding Telemetry to Robot Code / 在机器人代码中添加遥测

WPILib supports several different ways to record and send telemetry data from robot code.

WPILib 支持通过几种不同方式，在机器人代码中记录和发送遥测数据。

At the most basic level, the [RioLog](#) provides support for viewing print statements from robot code. This is useful for on-the-fly debugging of problematic code, but does not scale as console interfaces are not suitable for rich data streams.

最基本的方式是使用 [RioLog](#) 查看机器人代码输出的打印语句。这适合临时调试有问题的代码，但不适合扩展到大量数据，因为控制台界面不适合丰富的数据流。

WPILib supports several [dashboards](#) that allow users to more easily send rich telemetry data to the driver-station computer. All WPILib dashboards communicate with the [NetworkTables](#) protocol, and so they are to some degree interoperable (telemetry logged with one dashboard will be visible on the others, but the specific widgets/formatting will generally not be compatible). NetworkTables (and thus WPILib all dashboards) currently support the following data types:

WPILib 支持多个[仪表板](#)，使用户更方便地向 Driver Station 计算机发送丰富的遥测数据。所有 WPILib 仪表板都使用 [NetworkTables](#) 协议通信，因此它们在一定程度上可以互操作（一个仪表板记录的遥测数据可在其他仪表板上看到

，但具体小组件和格式通常并不兼容）。NetworkTables（因而也包括所有 WPILib 仪表板）目前支持以下数据类型：

- `boolean`
- `boolean[]`
- `double`
- `double[]`
- `string`
- `string[]`
- `byte[]`

Telemetry data can be sent to a WPILib dashboard using an associated WPILib method (for more details, see the documentation for the individual dashboard in question), or by [directly publishing to NetworkTables](#).

可以使用相应的 WPILib 方法向 WPILib 仪表板发送遥测数据（详情请参阅具体仪表板的文档），也可以[直接发布到 NetworkTables](#)。

While NetworkTables does not yet support serialization of complex data types (this is tentatively scheduled for 2024), mutable types from user code can be easily extended to interface directly with WPILib dashboards via the **Sendable** interface, whose usage is described in the next article.

虽然 NetworkTables 尚不支持复杂数据类型的序列化（暂定于 2024 年实现），但用户代码中的可变类型可以较容易地扩展，以通过 **Sendable** 接口直接与 WPILib 仪表板交互。下一篇文章将介绍其用法。

New for 2025: Robot Telemetry with Annotations (Java Only) / 2025 年新增：使用注解实现机器人遥测（仅 Java）

At its most basic, telemetry with annotations can be accomplished with only a few lines of code:

最基本的注解遥测只需几行代码即可实现：

1. Add the **@Logged** annotation to your main robot class and to the classes it references, like subsystems
1. 在主机器人类及其引用的类（例如 subsystem）上添加 **@Logged** 注解
2. Bind the logger to the robot in the constructor to start logging
2. 在构造函数中将 logger 绑定到机器人，以开始记录

This is enough to start logging data to NetworkTables for display and capture on a desktop application like a dashboard or AdvantageScope. Optionally, you can start the data log manager to capture the NetworkTables-logged data and save it to a log file on the roboRIO.

这样就足以开始向 NetworkTables 记录数据，以便在 dashboard 或 AdvantageScope 等桌面应用程序中显示和采集。也可以选择启动数据日志管理器，采集记录到 NetworkTables 的数据，并将其保存到 roboRIO 上的日志文件中。

```
@Logged
public class Robot extends TimedRobot {
    private final Arm arm;
    private final Drivetrain drivetrain;

    public Robot() {
        arm = new Arm();
        drivetrain = new Drivetrain();
        DataLogManager.start(); // Optional to mirror the NetworkTables-logged data to a file on disk
        Epilogue.bind(this);
    }
}

@Logged
public class Arm {
    // ...
}

@Logged
public class Drivetrain {
    // ...
}
```

The **@Logged** annotation, when placed on a class, tells the Java compiler to run WPILib's code generator to write custom logging code that will log everything it can in your class. The generated code will exist in the **build/generated/sources/annotationProcessor** folder in your robot project, and your code can reference and use that generated code. The **Epilogue** class is, in fact, one of those generated files - it is responsible for tracking all of the custom class-specific loggers created by the code generator and is the entry point for configuring and starting automatic logging. Placing the **@Logged** annotation on individual fields and methods will let you control how they appear in log files or networktables; placing **@NotLogged** on a field or method will explicitly exclude it from being logged, regardless of how its enclosing class is configured.

将 **@Logged** 注解放在类上，会指示 Java 编译器运行 WPILib 的代码生成器，生成自定义日志记录代码，尽可能记录该类中的所有内容。生成的代码位于机器人项目的 **build/generated/sources/annotationProcessor** 文件夹中，项目代码可以引用并使用它。实际上，**Epilogue** 类就是生成的文件之一，它负责跟踪代码生成器创建的所有特定于类的 logger，也是配置并启动自动日志记录的入口。在单个字段和方法上放置 **@Logged**

注解，可以控制它们在日志文件或 NetworkTables 中的显示方式；在字段或方法上放置 **@NotLogged**，则无论其所属类如何配置，都明确将其排除在日志记录之外。

Annotation logging will detect all public and private fields and all public methods in an annotated class, as long as those fields and methods return a known loggable type. Placing an **@Logged** annotation on any non-loggable field or method will result in a compilation error. Static fields and methods cannot be logged, nor can non-public methods or methods that accept arguments.

对于已标注的类，注解日志记录会检测所有 public 和 private 字段，以及所有 public 方法，前提是这些字段和方法返回已知的可记录类型。在不可记录的字段或方法上放置 **@Logged** 注解会导致编译错误。静态字段和方法不能记录，非 public 方法或接受参数的方法也不能记录。

Currently, annotation logging supports the following data types:

目前，注解日志记录支持以下数据类型：

- Any type (class, interface, or enum) tagged with the **@Logged** annotation
- 任何标有 **@Logged** 注解的类型（类、接口或枚举）
- Any type (class, interface, or enum) with a custom logger defined with a **@CustomLoggerFor** annotation (intended to be used for logging of third-party classes)
- 任何通过 **@CustomLoggerFor** 注解定义了自定义 logger 的类型（类、接口或枚举；用于记录第三方类）
- All primitive types (**byte**, **char**, **short**, **int**, **long**, **float**, **double**, and **boolean**) (note: this excludes wrapper types like **Byte**, **Integer**, and **Boolean**)
- 所有基本类型（**byte**、**char**、**short**、**int**、**long**、**float**、**double** 和 **boolean**；注意：不包括 **Byte**、**Integer** 和 **Boolean** 等包装类型）
- Strings
- 字符串
- Any type that inherits from **StructSerializable**, such as WPILib math types or custom user-defined types
- 任何继承自 **StructSerializable** 的类型，例如 WPILib 数学类型或用户自定义类型
- Any **edu.wpi.first.units.Measure** (logged as a double in terms of the base unit)
- 任何 **edu.wpi.first.units.Measure**（按基本单位记录为 double）
- byte[]**, **int[]**, **long[]**, **float[]**, **double[]**, and **boolean[]** primitive arrays
- byte[]**、**int[]**、**long[]**、**float[]**、**double[]** 和 **boolean[]** 基本类型数组
- String[]** arrays
- String[]** 数组
- Arrays of anything that inherits from **StructSerializable**, such as **Rotation2d[]** or **SwerveModuleState[]**
- 由继承自 **StructSerializable** 的对象构成的数组，例如 **Rotation2d[]** 或 **SwerveModuleState[]**
- Java collections, like **List** and **Set**, of **String** or **StructSerializable** objects, such as **Set<String>** or **List<SwerveModuleState>**
- 由 **String** 或 **StructSerializable** 对象组成的 Java 集合（例如 **List** 和 **Set**），如 **Set<String>** 或 **List<SwerveModuleState>**
- Any **Sendable** object, like a **DigitalInput** (note: commands and subsystems are excluded and will not be logged using their sendable implementation)
- 任何 **Sendable** 对象，例如 **DigitalInput**（注意：Command 和 subsystem 不包括在内，也不会通过其 Sendable 实现进行记录）

- **Mechanism2D** objects will not be logged as they do not implement the **Sendable** interface in a way that is compatible with annotation logging.
- **Mechanism2D** 对象不会被记录，因为它们实现 **Sendable** 接口的方式与注解日志记录不兼容。
- Any **BooleanSupplier**, **IntSupplier**, **LongSupplier**, or **DoubleSupplier** (note: this includes command triggers!)
- 任何 **BooleanSupplier**、**IntSupplier**、**LongSupplier** 或 **DoubleSupplier**（注意：这包括 Command trigger!）

Customizing your logging setup / 自定义日志记录设置

The default logging behavior is to log every non-static field (public or private) and public method in a class annotated with **@Logged**, where the associated log entries will use the name of the element. For example, a field declared as **private double m_someValue** will appear in logs under the entry "**m_someValue**", and a method declared as **public double getSomeValue()** will appear under the entry "**getSomeValue**".

默认的日志记录行为是：对于标有 **@Logged** 的类，记录其中的每个非静态字段（public 或 private）和 public 方法，并使用相应元素的名称作为日志条目名。例如，声明为 **private double m_someValue** 的字段会以条目名 "**m_someValue**" 出现在日志中；声明为 **public double getSomeValue()** 的方法则会以 "**getSomeValue**" 出现。

If you desire to only log certain fields or methods within a class, you can remove the **@Logged** annotation from the class itself and place it on individual fields/methods within the class. This allows you to opt-in on logging within a class instead of logging every value.

如果只想记录类中的特定字段或方法，可以从类本身移除 **@Logged**

注解，并将其放在各个字段或方法上。这样就可以选择性地记录类中的内容，而不是记录每一个值。

The names of log entries can be changed using the **name** configuration option on fields and methods (on a class-level annotation, it changes the name of the autogenerated logger class for avoiding name conflicts and does not change the log names of any instance of that class). For example, a **private double m_someValue** field with an **@Logged(name = "Motor Temperature")** will appear in logs as "**Motor Temperature**" instead of the default "**m_someValue**" that would be used.

可以使用字段和方法上的 **name** 配置选项更改日志条目的名称（放在类级注解上时，它会更改自动生成的 logger 类名称，以避免命名冲突，不会更改该类任何实例的日志名称）。例如，带有 **@Logged(name = "Motor Temperature")** 注解的 **private double m_someValue** 字段，会以 "**Motor Temperature**" 而不是默认的 "**m_someValue**" 出现在日志中。

@Logged Configurations / @Logged 配置

Attribute 属性	Description 说明
Name 名称	Sets a specific name to use for the annotated element. If placed on a class, this controls the name of the generated logger class for avoiding name conflicts. 设置用于被注解元素的特定名称。如果注解放在类上，此设置控制所生成 logger 类的名称，以避免命名冲突。

Attribute 属性	Description 说明
Importance 重要性	Sets the specific importance level for the annotated element, which can be used by the runtime minimumImportance configuration to control what data to log. If placed on a class, this sets the default importance level for all contained elements, which can be overridden on an element-by-element basis. Defaults to DEBUG. 设置被注解元素的具体重要性级别；运行时的 minimumImportance 配置可据此控制记录哪些数据。如果注解放在类上，此设置会为所有包含的元素设定默认重要性级别，各个元素仍可分别覆盖。默认为 DEBUG。
Logging Period 记录周期	Sets the amount of time between logging calls. 设置两次日志记录调用之间的时间间隔。
Logging Period Offset 记录周期偏移	Sets the offset of logging calls relative to the robot periodic functions. 设置日志记录调用相对于机器人周期函数的偏移。

Original code without logging / 未加入日志记录的原始代码

```
public class Robot extends RobotBase {
    private final Arm arm;

    public Robot() {
        arm = new Arm();
    }
}

class Arm {
    public final Trigger atLowStop = new Trigger(...);
    public final Trigger atHighStop = new Trigger(...);
    private Rotation2d lastPosition = getPosition();

    public Rotation2d getPosition() {
        // ...
    }

    public Measure<Velocity<Angle>> getSpeed() {
        // ...
    }
}
```

Code with logging (minimal) / 加入日志记录的代码（最简配置）

```
@Logged
public class Robot extends RobotBase {
    private final Arm arm; // Anything loggable within the arm object will be logged under an "arm" entry

    public Robot() {
        arm = new Arm();
        Epilogue.bind(this);
    }
}

@Logged
class Arm {
    public final Trigger atLowStop = new Trigger(...); // Logged as a boolean in an "atLowStop" entry
    public final Trigger atHighStop = new Trigger(...); // Logged as a boolean in an "atHighStop" entry
    private Rotation2d lastPosition = getPosition(); // Logged as a Rotation2d struct in a "lastPosition" entry

    // Logged as a Rotation2d struct object in a "getPosition" entry
    public Rotation2d getPosition() {
        // ...
    }

    // Logged as a double in terms of radians per second in a "getSpeed" entry
    public Measure<Velocity<Angle>> getSpeed() {
        // ...
    }
}
```

Data will be logged as:

数据将按以下形式记录：

```
/Robot/arm/atLowStop
/Robot/arm/atHighStop
/Robot/arm/lastPosition
/Robot/arm/getPosition
/Robot/arm/getSpeed
```

Code with logging (configured) / 加入日志记录的代码（自定义配置）

```
@Logged
public class Robot extends RobotBase {
    @Logged(name = "Arm")
    private Arm arm;

    public Robot() {
        arm = new Arm();
        DataLogManager.start();
        Epilogue.bind(this);
    }
}

class Arm {
    @Logged(name = "At Low Stop", importance = DEBUG)
    public final Trigger atLowStop = new Trigger(...);

    @Logged // defaults to an importance of DEBUG and the name as "atHighStop"
    public final Trigger atHighStop = new Trigger(...);

    @NotLogged // Redundant because the class strategy is opt-in
    private Rotation2d lastPosition = getPosition(); // No @Logged annotation, not logged

    @Logged(name = "Position", importance = CRITICAL)
    public Rotation2d getPosition() {
        // ...
    }

    @Logged
    public Measure<Velocity<Angle>> getSpeed() {
        // ...
    }
}
```

Data will be logged as:

数据将按以下形式记录：

```
/Robot/Arm/At Low Stop
/Robot/Arm/atHighStop
/Robot/Arm/Position
/Robot/Arm/getSpeed
```

The Epilogue Class / Epilogue 类

Epilogue is a special class that is created by the code generator. Its responsibility is to keep track of all the custom logger files and as the main entry point for configuring and starting automatic data logging.

Epilogue 是由代码生成器创建的特殊类。它负责跟踪所有自定义 logger 文件，并作为配置和启动自动数据日志记录的主要入口。

If your main robot class inherits from **TimedRobot**, the generated **Epilogue** class will have an additional **bind()** method that can be used to add a periodic logging call to the robot. The logging call will run at the same frequency as the main robot loop (typically 50Hz), but offset by half a phase to avoid extra CPU overhead while control loops are running. Keep in mind that, depending on your code structure, this means that logged data may be slightly out of sync with the data that the control loops operate on.

如果主机器人继承自 **TimedRobot**，生成的 **Epilogue** 类还会提供 **bind()** 方法，可用于为机器人添加周期性的日志记录调用。该调用与机器人主循环以相同频率运行（通常为 50Hz），但相位偏移半个周期，以避免控制循环运行期间产生额外 CPU 开销。请注意，具体取决于代码结构，这意味着记录的数据可能与控制循环使用的数据略有不同步。

Configuration 配置项	Description 说明	Default Value 默认值
Data Logger 数据 logger	The data logging implementation to use to save logged data. 用于保存所记录数据的数据日志记录实现。	NetworkTables-based logging, which the DataLogManager can read and save to a file on disk 基于 NetworkTables 的日志记 录, DataLogManager 可以读 取它并保存到磁盘上的文件
Root Data Entry 根数据条目	The root entry that logged data will appear under. Used for both NetworkTables-based and DataLog-based logging. Can be set to any string. 记录的数据所在的根条目。用于基于 NetworkTables 和基于 DataLog 的日志记录。可以设为任意字符串。	"Robot" "Robot"
Minimum Importance Level 最低重要性级别	The minimum importance level of data for it to be logged. Any logged data marked with a lower priority will be excluded from telemetry. 允许记录数据的最低重要性级别。标记为更低优先级的所有数据都将从遥 测中排除。	DEBUG (logs everything) DEBUG (记录所有内容)
Error handler 错误处理器	Catches and responds to errors encountered while logging data. Error handlers are used to prevent any errors encountered during logging from crashing the entire robot program. 捕获并处理记录数据时遇到的错误。错误处理器用于防止记录过程中的任 何错误使整个机器人程序崩溃。	Errors are caught and printed to the console 捕获错误并将其打印到控制台

```

@Logged
public class Robot extends TimedRobot {
    public Robot() {
        Epilogue.configure(config -> {
            // Log only to disk, instead of the default NetworkTables logging
            // Note that this means data cannot be analyzed in realtime by a dashboard
            config.backend = new FileBackend(DataLogManager.getLog());

            if (isSimulation()) {
                // If running in simulation, then we'd want to re-throw any errors that
                // occur so we can debug and fix them!
                config.errorHandler = ErrorHandler.crashOnError();
            }

            // Change the root data path
            config.root = "Telemetry";

            // Only log critical information instead of the default DEBUG level.
            // This can be helpful in a pinch to reduce network bandwidth or log file size
            // while still logging important information.
            config.minimumImportance = Logged.Importance.CRITICAL;
        });
        Epilogue.bind(this);
    }
}

```

Logging Third-Party Data / 记录第三方数据

A large part of the Java ecosystem is its access to third-party libraries that provide extra functionality, such as vendor libraries for interacting with custom hardware like motor controllers and sensors. If those vendors do not use logging annotations in their libraries, then you can still log them by writing a custom logger class! These classes are actually what the WPILib annotations will generate; you just write a custom one because third-party code cannot be annotated.

Java 生态系统的一个重要部分，是能够使用提供额外功能的第三方库，例如用于与电机控制器、传感器等自定义硬件交互的 vendor library。如果这些供应商没有在其库中使用日志记录注解，仍可以通过编写自定义 logger 类来记录相关数据！这些类实际上正是 WPILib 注解会生成的类；只是由于第三方代码无法添加注解，需要自己编写一个。

Custom loggers can be declared in any package, and only need to have the `@CustomLoggerFor` annotation present to be able to be detected and used. They must extend from `ClassSpecificLogger` (Javadoc) and must have a public no-argument constructor - failing to declare one will result in a compilation error - and must pass the type they log to the superclass constructor.

自定义 logger 可以在任何 package 中声明；只需带有 `@CustomLoggerFor` 注解，即可被检测和使用。它们必须继承 `ClassSpecificLogger` (Javadoc)，必须有一个 public 无参数构造函数——未声明会导致编译错误——并且必须将其记录的类型传给超类构造函数。

Note / 说明

Only one custom logger may be defined for a single type. Custom loggers will only be detected and used if they are defined in the robot project; custom loggers defined in third-party libraries cannot be detected.

每种类型只能定义一个自定义 logger。只有在机器人项目中定义的自定义 logger 才会被检测并使用；在第三方库中定义的自定义 logger 无法被检测。

```
class VendorMotor {
    public int getFaults();
    public void set(double speed);
    public double get();
    public double getAppliedVoltage();
    public double getInputCurrent();
}

@CustomLoggerFor(VendorMotor.class)
public class YourCustomVendorMotorLogger extends ClassSpecificLogger<VendorMotor> {
    public YourCustomVendorMotorLogger() {
        super(VendorMotor.class);
    }

    @Override
    public void update(EpilogueBackend backend, VendorMotor motor) {
        if (Epilogue.shouldLog(Logged.Importance.DEBUG)) {
            backend.log("Faults", motor.getFaults());
        }
        backend.log("Requested Speed (Duty Cycle)", motor.get());
        backend.log("Motor Voltage (V)", motor.getAppliedVoltage());
        backend.log("Input Current (A)", motor.getInputCurrent());
    }
}
```

Caveats and Limitations / 注意事项与限制

Because the logging code generation runs at compile time, information only available at runtime cannot be used. If a field is declared to be of some type **T**, then it will be logged using whatever configuration is defined for **T**, even if at runtime it is set to a concrete subtype **S** that has its own distinct logging configuration. You may encounter this if your code uses IO interfaces; consider annotating the interface with **@Logged** and provide methods for reading data you want to be logged regardless of implementation, such as motor voltages and sensor readings.

由于日志记录代码在编译时生成，无法使用仅在运行时才可获得的信息。如果某个字段被声明为类型 **T**，就会按照为 **T** 定义的配置记录，即使它在运行时被设为拥有独立日志记录配置的具体子类型 **S** 也是如此。使用 IO 接口的代码可能遇到这种情况；可考虑给接口添加 **@Logged** 注解，并提供读取所需日志数据的方法，使电机电压和传感器读数等数据无论采用何种实现都能被记录。

Null values are unrepresentable in log data and therefore cannot be logged. If a logged field is set to **null** or is uninitialized, or if a logged method returns **null**, then the logging code will throw a **NullPointerException** to be handled by the configured error handler.

日志数据无法表示 null 值，因此不能记录 null。如果被记录的字段被设为 **null** 或尚未初始化，或者被记录的方法返回 **null**，日志记录代码会抛出 **NullPointerException**，并由配置的错误处理器处理。

Data logging increases CPU load on the roboRIO and can lead to loop overruns. The CPU load is not caused by the logging itself, but from calling expensive methods to read data. This is most often seen when querying information from devices on a CAN bus. If your code frequently sees loop overruns caused by logging, consider reducing the number of logged fields or restructure your code to periodically read and cache data from connected devices and log the cached data instead of logging methods that needlessly re-query it.

数据日志记录会增加 roboRIO 的 CPU 负载，并可能导致循环超时。CPU 负载并非来自日志记录操作本身，而是来自调用读取数据的高开销方法。这种情况最常见于查询 CAN 总线上设备的信息。如果代码频繁因日志记录而出现循环超时，可考虑减少记录的字段数量，或调整代码结构，定期读取并缓存连接设备的数据，再记录缓存数据，而不要记录会重复查询设备的方法。

Note / 说明

Time spent on logging in each loop will be logged to **/Epilogue/Stats/Last Run** in NetworkTables and can be analyzed in a tool like AdvantageScope.

每次循环用于日志记录的时间会记录到 NetworkTables 的 **/Epilogue/Stats/Last Run**，可以使用 AdvantageScope 等工具分析。

Recording Faults with Persistent Alerts / 使用持久警报记录故障

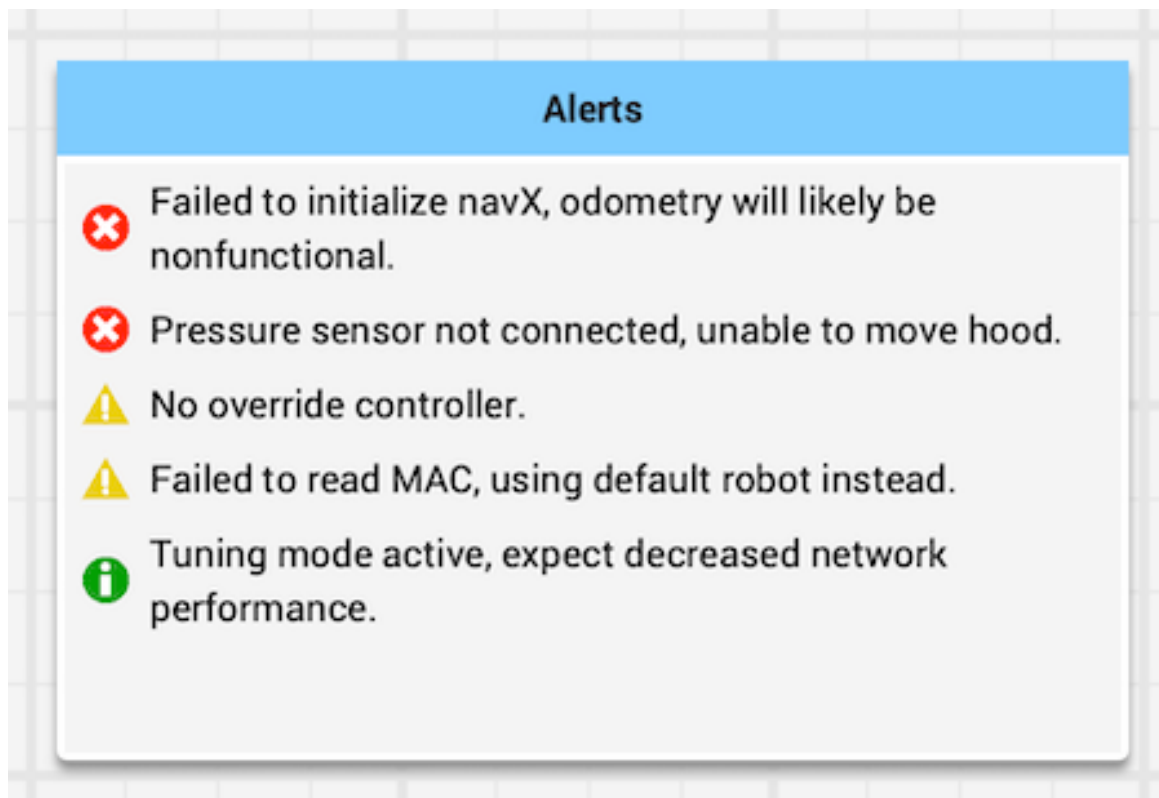
Robots encounter a variety of fault conditions: disconnected sensors/motors, invalid mechanism states, initialization failures, etc. While the FRC Driver Station provides a [console interface](#) for instantaneous alerts, the risk of missing important messages makes it poorly suited to communicate faults that persist over time.

机器人会遇到各种故障状态，例如传感器或电机断开连接、机构状态无效、初始化失败等。虽然 FRC Driver Station 提供用于即时警报的[控制台界面](#)，但重要消息可能被错过，因此它不适合传达持续存在的故障。

Instead, the **Alert** class ([Java](#)) can be used for managing persistent alerts published via Network Tables. Alerts are assigned a priority (error, warning, or info) and can be activated or deactivated in robot code. The set of active alerts can be displayed on a dashboard, as shown below in Shuffleboard.

可以改用 **Alert** 类，管理通过 NetworkTables

发布的持久警报。警报会分配优先级（错误、警告或信息），并可在机器人代码中激活或停用。如下面的 Shuffleboard 截图所示，仪表板可显示当前激活的警报。



A screenshot of the alerts widget in Shuffleboard, with several active alerts. / Shuffleboard
中警报小组件的截图，其中有多条已激活的警报。

Active alerts are automatically displayed in order based on priority and how recently they were activated, with newer and more critical alerts at the top of the list. This provides a mechanism for drivers (on the field) and programmers (off the field) to quickly assess the state of the robot and determine whether it is in a “match-ready” state. Highly descriptive alerts also allow drive teams to adapt to faults in real time, such as switching strategies in case of a subsystem failure.

激活的警报会根据优先级和最近激活时间自动排序，越新且越严重的警报排得越靠前。这让场上的驾驶员和场外的程序员能够快速评估机器人状态，判断其是否“准备好比赛”。描述充分的警报还使驾驶队伍能实时应对故障，例如在 subsystem 发生故障时切换策略。

API Usage / API 用法

Alerts are created and managed using the **Alert** class. Typically, an alert should be instantiated once and stored as a class member. The **set** method accepts a boolean for the alert state, which activates or deactivates the alert. Change detection is built-in, so calling **set** multiple times with the same value does not affect the order in which alerts are displayed.

使用 **Alert** 类创建和管理警报。通常，每个警报只应创建一次，并作为类成员保存。**set** 方法接受表示警报状态的布尔值，用于激活或停用警报。此类内置了变化检测，因此多次以相同值调用 **set** 确实会影响警报显示顺序。译注：原文的 “does affect” 与前述 “变化检测” 含义相矛盾，疑为 “does not affect”。

Alert states are often expressed most easily as a conditional, such as whether the latest signal from a CAN device is up-to-date. In this case, we recommend calling the **set** method in a periodic loop and relying on the built-in change detection to activate the alert.

警报状态往往最容易表示为一个条件，例如 CAN 设备的最新信号是否仍是最新状态。在这种情况下，建议在 periodic 循环中调用 **set** 方法，并依靠内置的变化检测来激活警报。

```
class Robot {
  Alert alert = new Alert("Something went wrong", AlertType.kWarning);

  periodic() {
    alert.set(...);
  }
}
```

Note / 说明

Suggested usage of each alert type (error, warning, or info) is provided in the enum API documentation.

各警报类型（错误、警告或信息）的建议用途见枚举 API 文档。

The text description of an alert can be updated after the alert has been created by calling **setText**, allowing additional information to be provided inline. For example, an alert indicating that a motor is overheating could include the motor's current temperature.

创建警报后，可以调用 **setText**

更新其文字描述，以便在其中提供更多信息。例如，表示电机过热的警报可以包含该电机的当前温度。

Optionally, a group name can also be provided to the **Alert** constructor. Alerts are published to Network Tables using this name, allowing multiple sets of alerts to be displayed on separate widgets (such as “Auto” and “Teleop” alert sets). If the group name is omitted, the alert is published to a group called “Alerts”.

也可以选择向 **Alert** 构造函数传入组名。警报会用这个名称发布到 NetworkTables，使多个警报组可显示在不同小组件中（例如 “Auto” 和 “Teleop” 警报组）。如果省略组名，警报就会发布到名为 “Alerts” 的组。

Dashboard Usage / 仪表板用法

Alerts are published to Network Tables using the key **/SmartDashboard/<Group Name>**. Shuffleboard automatically populates data from this table to the “SmartDashboard” tab, or the widget can be manually created by dragging the alert table from the Network Tables tree to the main view as described [here](#).

警报通过键 `/SmartDashboard/<Group Name>` 发布到 NetworkTables。Shuffleboard 会自动将此表中的数据填充到 “SmartDashboard” 标签页；也可以按[这里](#)所述，将 NetworkTables 树中的警报表拖到主视图，手动创建小组件。

Note / 说明

In addition to Shuffleboard, the alerts API is supported by many third-party dashboards. Check the documentation of your preferred dashboard for details.

除 Shuffleboard 外，许多第三方仪表板也支持 Alerts API。详情请查阅你所使用仪表板的文档。

Robot Telemetry with Sendable / 使用 Sendable 实现机器人遥测

While the WPILib dashboard APIs allow users to easily send small pieces of data from their robot code to the dashboard, it is often tedious to manually write code for publishing telemetry values from the robot code's operational logic.

WPILib 仪表板 API 使用户能够轻松地从机器人代码向仪表板发送少量数据，但在机器人代码的实际运行逻辑中，手动编写发布遥测值的代码通常很繁琐。

A cleaner approach is to leverage the existing object-oriented structure of user code to mark important data fields for telemetry logging in a [declarative programming](#) style. The WPILib framework can then handle the tedious/tricky part of correctly reading from (and, potentially, writing to) those fields for you, greatly reducing the total amount of code the user has to write and improving readability.

更简洁的方法是利用用户代码现有的面向对象结构，以[声明式编程](#)方式标记重要的数据字段，以便记录遥测数据。WPILib 框架随后可以代为正确读取这些字段（必要时也可以写入），处理繁琐而容易出错的部分，从而大幅减少用户需要编写的代码，并提高可读性。

WPILib provides this functionality with the **Sendable** interface. Classes that implement **Sendable** are able to register value listeners that automatically send data to the dashboard - and, in some cases, receive values back. These classes can be declaratively sent to any of the WPILib dashboards (as one would an ordinary data field), removing the need for teams to write their own code to send/poll for updates.

WPILib 通过 **Sendable** 接口提供此功能。实现 **Sendable** 的类可以注册值监听器，自动向仪表板发送数据；在某些情况下，也可以从仪表板接收值。这些类可以像普通数据字段一样，以声明式方式发送到任意 WPILib 仪表板，使队伍无需自己编写发送数据或轮询更新的代码。

What is Sendable? / 什么是 Sendable?

Sendable (Java) is an interface provided by WPILib to facilitate robot telemetry. Classes that implement **Sendable** can declaratively send their state to the dashboard - once declared, WPILib will automatically send the telemetry values every robot loop. This removes the need for teams to handle the iteration-to-iteration logic of sending and receiving values from the dashboard, and also allows teams to separate their telemetry code from their robot logic.

Sendable 接口由 WPILib 提供，用于简化机器人遥测。实现 **Sendable** 的类可以声明式地将自身状态发送到仪表板；声明后，WPILib 会在机器人的每个循环中自动发送遥测值。这样，队伍无需处理每个循环之间的数据发送和接收逻辑，也能将遥测代码与机器人逻辑分开。

Many WPILib classes (such as [Commands](#)) already implement **Sendable**, and so can be sent to the dashboard without any user modification. Users are also able to easily extend their own classes to implement **Sendable**.

许多 WPILib 类（例如 [Command](#)）已经实现

Sendable，因此无需用户修改即可发送到仪表板。用户也可以轻松扩展自己的类来实现 **Sendable**。

The **Sendable** interface contains only one method: **initSendable**. Implementing classes override this method to perform the binding of in-code data values to structured [JSON](#) data, which is then automatically sent to the robot dashboard via NetworkTables. Implementation of the **Sendable** interface is discussed in the [next article](#).

Sendable 接口只有一个方法：**initSendable**。实现类通过重写此方法，将代码中的数据值绑定到结构化的 [JSON](#) 数据，随后这些数据会通过 NetworkTables 自动发送到机器人仪表板。下一篇文章将讨论如何实现 **Sendable** 接口。

Sending a Sendable to the Dashboard / 将 Sendable 发送到仪表板

Note / 说明

Unlike simple data types, Sendables are automatically kept up-to-date on the dashboard by WPILib, without any further user code - “set it and forget it”. Accordingly, they should usually be sent to the dashboard in an initialization block or constructor, not in a periodic function.

与简单数据类型不同，WPILib 会自动保持仪表板上的 Sendable 为最新状态，无需用户编写其他代码——“设好就不用再管”。因此，通常应在初始化代码块或构造函数中将其发送到仪表板，而不是在 periodic 函数中发送。

To send a **Sendable** object to the dashboard, simply use the dashboard’s **putData** method. For example, an “arm” class that uses a **PID Controller** can automatically log telemetry from the controller by calling the following in its constructor:

要把 **Sendable** 对象发送到仪表板，只需使用仪表板的 **putData** 方法。例如，在使用 **PID Controller** 的“机械臂”类中，可在其构造函数中调用以下代码，自动记录控制器的遥测数据：

```
SmartDashboard.putData("Arm PID", armPIDController);
```

Additionally, some **Sendable** classes bind setters to the data values sent from the dashboard to the robot, allowing remote tuning of robot parameters.

此外，有些 **Sendable** 类会将 setter 绑定到从仪表板发送到机器人的数据值，从而允许远程调校机器人参数。

On-Robot Telemetry Recording Into Data Logs / 将机器人端遥测数据记录到数据日志中

By default, no telemetry data is recorded (saved) on the robot. The **DataLogManager** class provides a convenient wrapper around the lower-level **DataLog** class for on-robot recording of telemetry data into data logs. The WPILib data logs are binary for size and speed reasons. In general, the data log facilities provided by WPILib have minimal overhead to robot code, as all file I/O is performed on a separate thread—the log operation consists of mainly a mutex acquisition and copying the data.

默认情况下，机器人不会记录（保存）任何遥测数据。**DataLogManager** 类是底层 **DataLog** 类的便利封装，用于在机器人上将遥测数据记录到数据日志中。出于文件大小和速度的考虑，WPILib 数据日志采用二进制格式。通常，WPILib 提供的数据日志功能对机器人代码的额外开销很小，因为所有文件 I/O 都在单独的线程上执行；日志操作主要包括获取互斥锁和复制数据。

Structure of Data Logs / 数据日志的结构

Similar to NetworkTables, data logs have the concept of entries with string identifiers (keys) with a specified data type. Unlike NetworkTables, the data type cannot be changed after the entry is created, and entries also have metadata—an arbitrary (but typically JSON) string that can be used to convey additional information about the entry such as the data source or data schema. Also unlike NetworkTables, data log operation is unidirectional—the **DataLog** class can only write data logs (it does not support read-back of written values) and the **DataLogReader** class can only read data logs (it does not support changing values in the data log).

与 NetworkTables 类似，数据日志也有条目的概念，每个条目具有字符串标识符（键）和指定的数据类型。与 NetworkTables 不同，条目创建后不能更改数据类型，而且条目还包含元数据：一个任意的字符串（通常是 JSON），可用于传达条目的额外信息，例如数据来源或数据结构。数据日志的操作也与 NetworkTables 不同，它是单向的：**DataLog** 类只能写入数据日志（不支持回读已写入的值），**DataLogReader** 类只能读取数据日志（不支持修改日志中的值）。

Data logs consist of a series of timestamped records. Control records allow starting, finishing, or changing the metadata of entries, and data records record data value changes. Timestamps are stored in integer microseconds; when running on the RoboRIO, the FPGA timestamp is used (the same timestamp returned by **Timer.getFPGATimestamp()**).

数据日志由一系列带时间戳的记录组成。控制记录可以启动或结束条目，也可以更改条目的元数据；数据记录则记录数据值的变化。时间戳以整数微秒存储；在 roboRIO 上运行时使用 FPGA 时间戳（与 **Timer.getFPGATimestamp()** 返回的时间戳相同）。

Note / 说明

For more information on the details of the data log file format, see the [WPILib Data Log File Format Specification](#).
有关数据日志文件格式的更多细节，请参阅 [WPILib 数据日志文件格式规范](#)。

Standard Data Logging using DataLogManager / 使用 DataLogManager 进行标准数据记录

The **DataLogManager** class (Java) provides a centralized data log that provides automatic data log file management. It automatically cleans up old files when disk space is low and renames the file based either on current date/time or (if available) competition match number. The data file will be saved to a USB flash drive in a folder called **logs** if one is attached, or to **/home/lvuser/logs** otherwise.

DataLogManager 类提供集中式数据日志，并自动管理日志文件。磁盘空间不足时，它会自动清理旧文件，并根据当前日期/时间或（如果可用）比赛场次编号重命名文件。若插入了 USB 闪存盘，数据文件会保存在其中名为 **logs** 的文件夹；否则保存在 **/home/lvuser/logs**。

Note / 说明

USB flash drives need to be formatted as FAT32 to work with the roboRIO. NTFS or exFAT formatted drives will not work. Flash drives of 32GB or smaller are recommended, as Windows doesn't format drives larger than 32GB with FAT32.

USB 闪存盘需要格式化为 FAT32，才能与 roboRIO 配合使用。NTFS 或 exFAT 格式的闪存盘无法使用。建议使用容量为 32GB 或更小的闪存盘，因为 Windows 不会将大于 32GB 的闪存盘格式化为 FAT32。

Log files are initially named **FRC_TBD_{random}.wpilog** until the DS connects. After the DS connects, the log file is renamed to **FRC_YYYYMMdd_HHmmss.wpilog** (where the date/time is UTC). If the **FMS** is connected and provides a match number, the log file is renamed to **FRC_YYYYMMdd_HHmmss_{event}_{match}.wpilog**.

日志文件最初命名为 **FRC_TBD_{random}.wpilog**，直到 DS 连接。DS 连接后，文件重命名为 **FRC_YYYYMMdd_HHmmss.wpilog**（日期/时间采用 UTC）。如果 **FMS** 已连接并提供比赛场次编号，文件会重命名为 **FRC_YYYYMMdd_HHmmss_{event}_{match}.wpilog**。

On startup, all existing log files where a DS has not been connected will be deleted. If there is less than 50 MB of free space on the target storage, **FRC_** log files are deleted (oldest to newest) until there is 50 MB free OR there are 10 files remaining.

启动时，所有尚未连接过 DS 的现有日志文件都会被删除。如果目标存储空间的剩余容量小于 50 MB，系统会按从旧到新的顺序删除以 **FRC_** 开头的日志文件，直到剩余空间达到 50 MB，或者只剩 10 个文件。

The most basic usage of DataLogManager only requires a single line of code (typically this would be called from **Robot** constructor). This will record all NetworkTables changes to the data log.

DataLogManager 最基本的用法只需要一行代码（通常在 **Robot** 构造函数中调用）。这会将所有 NetworkTables 变更记录到数据日志中。

```
import edu.wpi.first.wpilibj.DataLogManager;
// Starts recording to data log
DataLogManager.start();
```

DataLogManager provides a convenience function (**DataLogManager.log()**) for logging of text messages to the **messages** entry in the data log. The message is also printed to standard output, so this can be a replacement for **System.out.println()**.

DataLogManager 提供便利函数 **DataLogManager.log()**，用于将文本消息写入数据日志的 **messages** 条目。消息也会打印到标准输出，因此可用来代替 **System.out.println()**。

DataLogManager also records the current roboRIO system time (in UTC) to the data log every ~5 seconds to the **systemTime** entry in the data log. This can be used to (roughly) synchronize the data log with other records such as DS logs or match video.

DataLogManager 还会约每 5 秒将当前 roboRIO 系统时间（UTC）记录到数据日志的 **systemTime** 条目。这可用于将数据日志与 DS 日志或比赛视频等其他记录进行（粗略）同步。

For custom logging, the managed **DataLog** can be accessed via **DataLogManager.getLog()**.

对于自定义日志记录，可通过 **DataLogManager.getLog()** 访问受管理的 **DataLog**。

Logging Joystick Data / 记录摇杆数据

DataLogManager by default does not record joystick data. The **DriverStation** class provides support for logging of DS control and joystick data via the **startDataLog()** function:

默认情况下，**DataLogManager** 不记录摇杆数据。**DriverStation** 类通过 **startDataLog()** 函数支持记录 DS 控制数据和摇杆数据：

```
import edu.wpi.first.wpilibj.DataLogManager;
import edu.wpi.first.wpilibj.DriverStation;
// Starts recording to data log
DataLogManager.start();
// Record both DS control and joystick data
DriverStation.startDataLog(DataLogManager.getLog());
// (alternatively) Record only DS control data
DriverStation.startDataLog(DataLogManager.getLog(), false);
```

Custom Data Logging using DataLog / 使用 DataLog 自定义记录数据

The **DataLog** class (Java) and its associated LogEntry classes (e.g. **BooleanLogEntry**, **DoubleLogEntry**, etc) provides low-level access for writing data logs.

DataLog 类及其关联的 LogEntry 类（例如 **BooleanLogEntry**、**DoubleLogEntry** 等）提供写入数据日志的底层接口。

Note / 说明

Unlike NetworkTables, there is no change checking performed. Every call to a **LogEntry.append()** function will result in a record being written to the data log. Checking for changes and only appending to the log when necessary is the responsibility of the caller.

与 NetworkTables 不同，这里不会执行变化检测。每次调用 **LogEntry.append()** 函数都会向数据日志写入一条记录。调用者有责任检查数据是否变化，并仅在必要时追加到日志中。

The LogEntry classes can be used in conjunction with DataLogManager to record values only to a data log and not to NetworkTables:

LogEntry 类可以与 DataLogManager 配合使用，仅把数值记录到数据日志，而不发布到 NetworkTables：

```

import edu.wpi.first.util.datalog.BooleanLogEntry;
import edu.wpi.first.util.datalog.DataLog;
import edu.wpi.first.util.datalog.DoubleLogEntry;
import edu.wpi.first.util.datalog.StringLogEntry;
import edu.wpi.first.wpilibj.DataLogManager;
BooleanLogEntry myBooleanLog;
DoubleLogEntry myDoubleLog;
StringLogEntry myStringLog;

public Robot() {
    // Starts recording to data log
    DataLogManager.start();
    // Set up custom log entries
    DataLog log = DataLogManager.getLog();
    myBooleanLog = new BooleanLogEntry(log, "/my/boolean");
    myDoubleLog = new DoubleLogEntry(log, "/my/double");
    myStringLog = new StringLogEntry(log, "/my/string");
}

public void teleopPeriodic() {
    if (...) {
        // Only log when necessary
        myBooleanLog.append(true);
        myDoubleLog.append(3.5);
        myStringLog.append("wow!");
    }
}
}

```

Downloading & Processing Data Logs / 下载和处理数据日志

Data logs can be processed and viewed offline by AdvantageScope, the DataLogTool, or custom utilities. If data log files are being stored to the roboRIO integrated flash memory instead of a removable USB flash drive, it's important to periodically download and delete data logs to avoid the storage from filling up.

可以使用 AdvantageScope、DataLogTool 或自定义工具离线处理和查看数据日志。如果数据日志文件保存在 roboRIO 的集成闪存中，而不是可移除的 USB 闪存盘上，就必须定期下载并删除日志，以免存储空间耗尽。

Managing Data Logs with AdvantageScope / 使用 AdvantageScope 管理数据日志

AdvantageScope is an analysis tool that supports downloading, visualizing, and exporting data logs. See the relevant sections of the documentation for more details:

AdvantageScope 是支持下载、可视化和导出数据日志的分析工具。详情请参阅文档的相关章节：

- [Downloading log files](#)
- [下载日志文件](#)
- [Exporting to new formats](#)
- [导出为其他格式](#)

Managing Data Logs with the DataLogTool / 使用 DataLogTool 管理数据日志

The DataLogTool desktop application integrates a SFTP client for downloading data log files from a network device (e.g. roboRIO or coprocessor) to the local computer.

DataLogTool 桌面应用集成了 SFTP 客户端，用于从网络设备（例如 roboRIO 或协处理器）将数据日志文件下载到本地计算机。

This process consists of four steps:

此过程包含四个步骤：

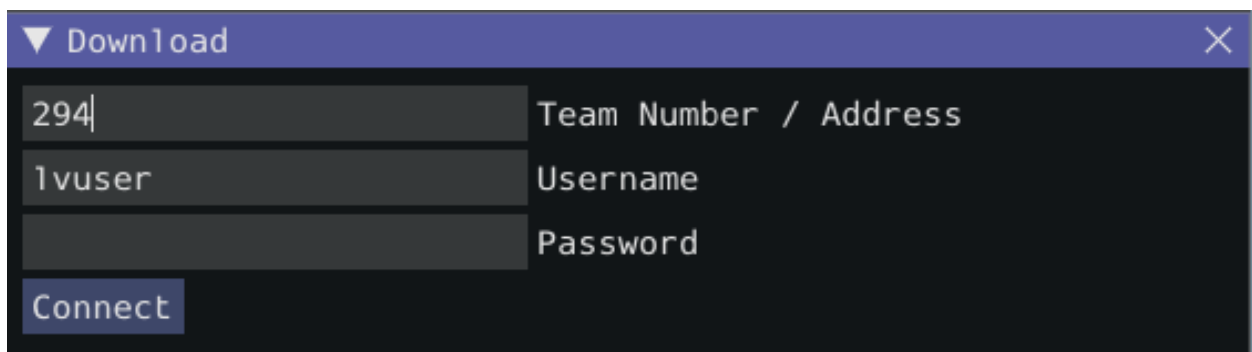
1. Connect to roboRIO or coprocessor
1. 连接到 roboRIO 或协处理器
2. Navigate to remote directory and select what files to download
2. 浏览远程目录并选择要下载的文件
3. Select download folder
3. 选择下载文件夹
4. Download files and optionally delete remote files after downloading
4. 下载文件，并可选择在下载后删除远程文件

Connecting to RoboRIO / 连接到 RoboRIO

Note / 说明

The downloader uses SSH, so it will not be able to connect wirelessly if the radio firewall is enabled (e.g. when the robot is on the competition field).

下载器使用 SSH，因此如果无线电防火墙已启用（例如机器人位于比赛场地上），就无法通过无线网络连接。



Connection display showing team number, username, and password fields. / 连接界面，显示队伍编号、用户名和密码字段。

Either a team number, IP address, or hostname can be entered into the Team Number / Address field. This field specifies the remote host to connect to. If a team number is entered, **roborio-TEAM-frc.local** is used as the connection address.

可以在 Team Number / Address 字段中输入队伍编号、IP 地址或主机名。此字段指定要连接的远程主机。输入队伍编号时，连接地址使用 **roborio-TEAM-frc.local**。

The remote username and password are also entered here. For the roboRIO, the username should be **lvuser** with a blank password.

这里还需输入远程用户名和密码。对于 roboRIO，用户名应为 **lvuser**，密码留空。

The tool also supports connecting to network devices other than the roboRIO, such as coprocessors, as long as the device supports SFTP password-based authentication.

此工具还支持连接到 roboRIO 以外的网络设备，例如协处理器，只要设备支持基于密码的 SFTP 身份验证即可。

Click Connect to connect to the remote device. This will attempt to connect to the device. The connection attempt can be aborted at any time by clicking Disconnect. If the application is unable to connect to the remote device, an error will be displayed above the Team Number / Address field and a new connection can be attempted.

单击 Connect 连接远程设备。工具会尝试建立连接。随时可以单击 Disconnect 中止连接尝试。如果应用无法连接到远程设备，Team Number / Address 字段上方会显示错误，之后可以再次尝试连接。

Downloading Files / 下载文件

After the connection is successfully established, a simplified file browser will be displayed. This is used to navigate the remote filesystem and select which files to download. The first text box shows the current directory. A specific directory can be navigated to by typing it in this text box and pressing Enter. Alternatively, directory navigation can be performed by clicking on one of the directories that are listed below the remote dir textbox. Following the list of directories is a table of files. Only files with a **.wpilog** extension are shown, so the table will be empty if there are no log files in the current directory. The checkbox next to each data log file indicates whether the file should be downloaded.

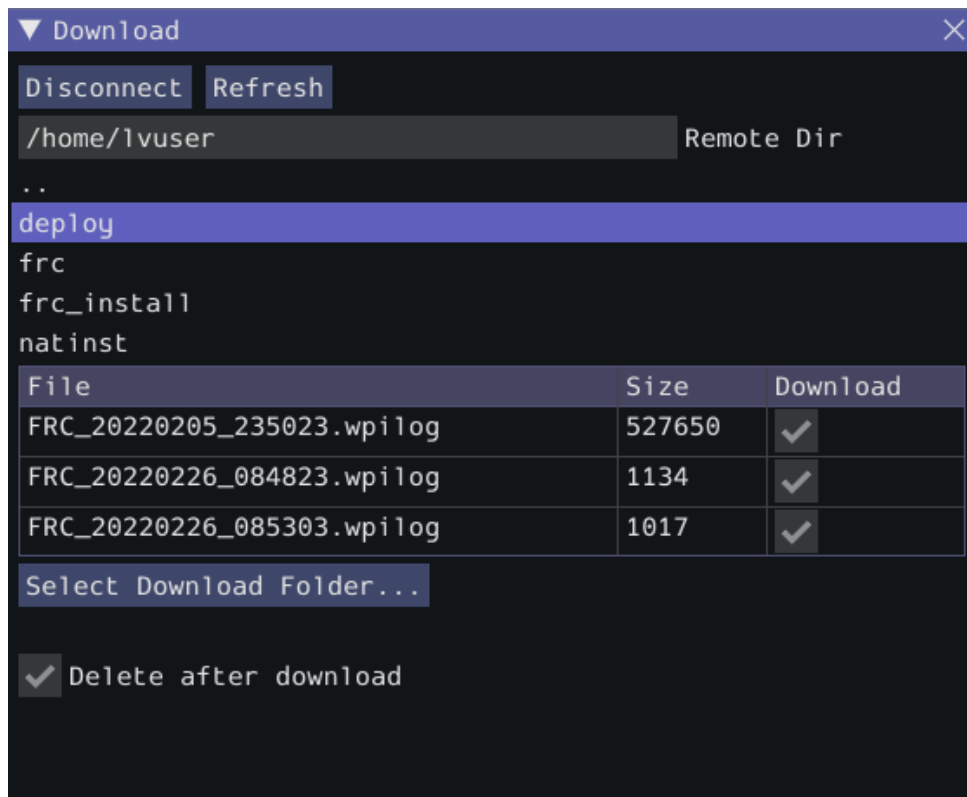
连接成功后，会显示一个简化的文件浏览器，用于浏览远程文件系统并选择要下载的文件。第一个文本框显示当前目录。可以在文本框中输入具体目录并按

Enter，跳转到该目录；也可以单击下方列出的目录进行浏览。目录列表之后是文件表格。这里仅显示扩展名为 **.wpilog**

的文件，因此如果当前目录没有日志文件，表格就会为空。每个数据日志文件旁的复选框表示是否下载该文件。

Note / 说明

On the roboRIO, log files are typically saved to either `/home/lvuser/logs` or `/u/logs` (USB stick location).
在 roboRIO 上，日志文件通常保存在 `/home/lvuser/logs` 或 `/u/logs` (USB 闪存盘位置)。



Remote file browser showing remote directory, list of directories, and list of files with checkboxes next to each one. / 远程文件浏览器，显示远程目录、目录列表，以及每个文件旁有复选框的文件列表。

Click Select Download Folder... to bring up a file browser for the local computer.

单击 Select Download Folder...，打开本地计算机的文件浏览器。

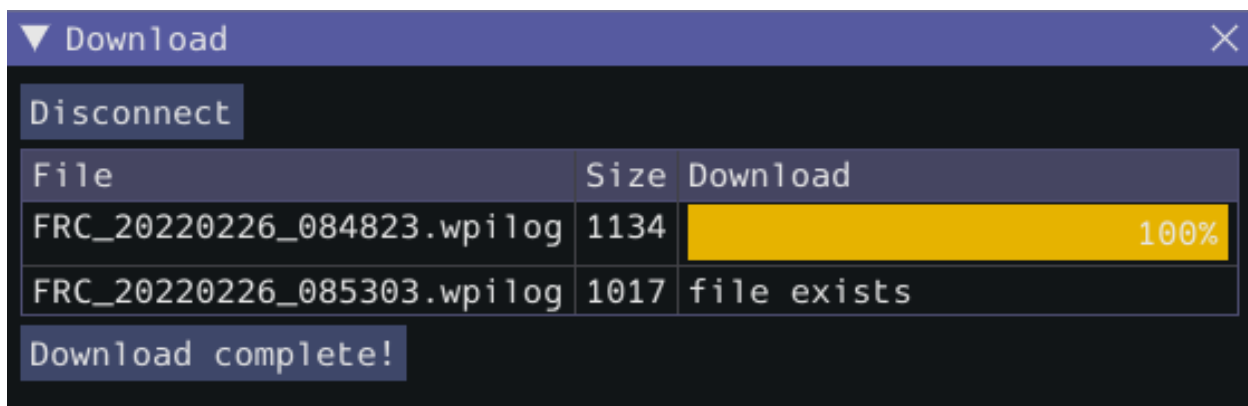
If you want to delete the files from the remote device after they are downloaded, check the Delete after download checkbox.

如果希望下载后从远程设备中删除文件，请勾选 Delete after download 复选框。

Once a download folder is selected, Download will appear. After clicking this button, the display will change to a download progress display. Any errors will be shown next to each file. Click Download complete! to return to the file browser.

选择下载文件夹后，Download

按钮会出现。单击后，界面会切换为下载进度显示。每个文件旁都会显示相应错误。单击 Download complete! 返回文件浏览器。

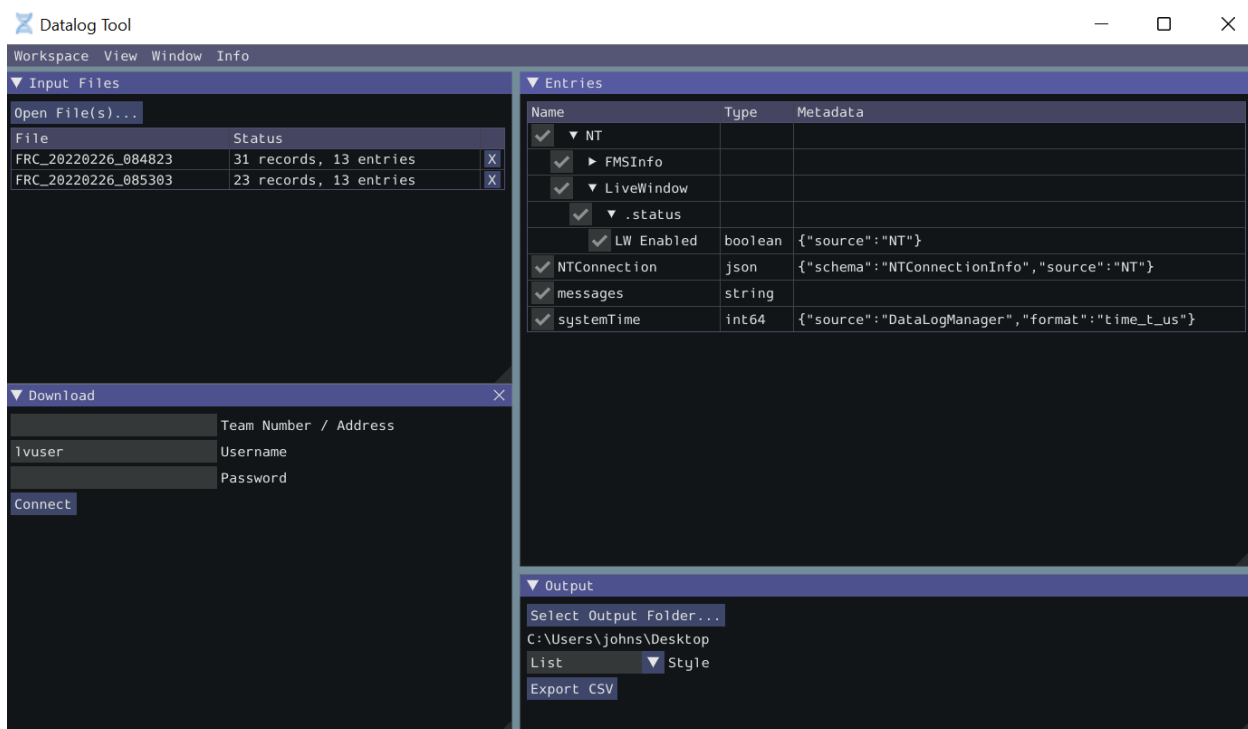


Download status showing 100% completion on the first file and a "file exists" error on the second file. / 下载状态：第一个文件显示完成 100%，第二个文件显示 “file exists” 错误。

Converting Data Logs to CSV / 将数据日志转换为 CSV

As data logs are binary files, the DataLogTool desktop application provides functionality to convert data logs into CSV files for further processing or analysis. Multiple data logs may be simultaneously loaded into the tool for batch processing, and partial data exports can be performed by selecting only the data that is desired to be output.

数据日志是二进制文件，因此 DataLogTool 桌面应用提供将其转换为 CSV 文件的功能，以便进一步处理或分析。可以同时多个数据日志加载到工具中进行批量处理，也可以只选择需要导出的数据，执行部分数据导出。



DataLogTool window showing two open files, a tree view of entries with corresponding checkboxes and information about each entry, and an output window with export style option. / DataLogTool 窗口显示两个已打开的文件、包含条目及其复选框和对应信息的树形视图，以及带有导出样式选项的输出窗口。

The conversion process is started by opening data log files in the “Input Files” window. Files are opened by clicking Open File(s)⋮. Summary status on each file (e.g. number of records and entries) is displayed. Clicking X in the table row closes the file.

转换过程从在“Input Files”窗口中打开数据日志文件开始。单击 Open File(s)… 打开文件。界面会显示每个文件的概要状态（例如记录数和条目数）。单击表格行中的 X 可关闭文件。

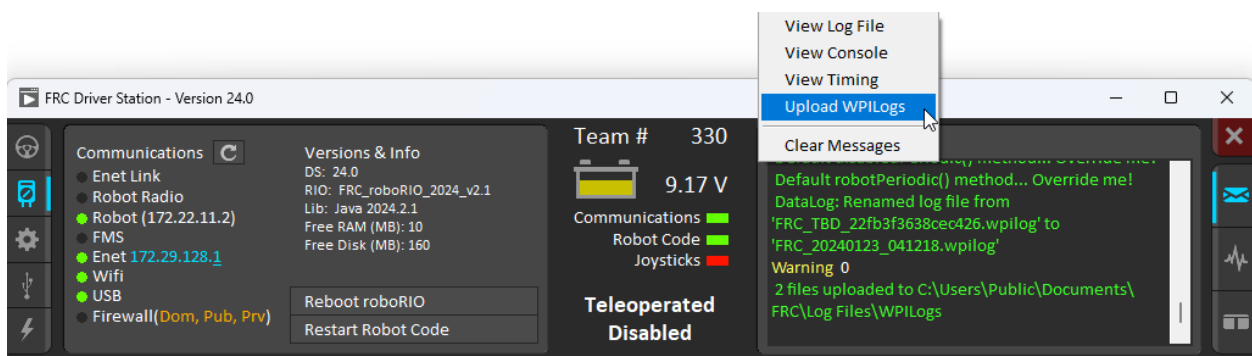
After at least one file is loaded, the “Entries” window displays a tree view of the entries (this can be changed to a flat view by right clicking on the “Entries” window title bar and unchecking Tree View). Individual entries or entire subtrees can be checked or unchecked to indicate whether they should be included in the export. The data type information and initial metadata for each entry is also shown in the table. As the “Entries” view shows a merged view of all entries across all input files, if more than one input file is open, hovering over an entry’s name will highlight what input files contain that entry.

至少加载一个文件后，“Entries”窗口会以树形视图显示条目（右键单击“Entries”窗口标题栏并取消勾选 Tree View，可切换为平铺视图）。可以勾选或取消勾选单个条目或整个子树，以指定导出时是否包含它们。表格还会显示每个条目的数据类型信息和初始元数据。由于“Entries”视图显示所有输入文件中条目的合并视图，如果打开了多个输入文件，将鼠标悬停在条目名称上时，会突出显示包含该条目的输入文件。

The output window is used to specify the output folder (via Select Output Folder…) as well as the output style (list or table). The list output style outputs a CSV file with 3 columns (timestamp, entry name, and value) and a row for every value change (for every exported entry). The table output style outputs a CSV file with a timestamp column and a column for every exported entry; a row is output for every value change (for every exported entry), but the value is placed in the correct column for that entry. Clicking Export CSV will create a .csv file in the output folder corresponding to each input file.

输出窗口用于指定输出文件夹（通过 Select Output Folder…），以及输出样式（列表或表格）。列表样式输出包含三列（时间戳、条目名称和值）的 CSV 文件，每次数值变化（对于每个导出的条目）占一行。表格样式输出包含一列时间戳和每个导出条目各一列的 CSV 文件；每次数值变化（对于每个导出的条目）也输出一行，但数值会放入该条目的正确列。单击 Export CSV，会在输出文件夹中为每个输入文件创建一个对应的 .csv 文件。

Managing Data Logs with the Driver Station / 使用 Driver Station 管理数据日志



Selecting Upload WPILogs in the Driver Station / 在 Driver Station 中选择 Upload WPILogs

The Driver Station software can download WPILogs. Click on the gear icon and select Upload WPILogs. The logs in `/home/lvuser/logs` or `/u/logs` will be downloaded automatically to `C:\Users\Public\Documents\FRC\Log Files\WPILogs`

Driver Station 软件可以下载 WPILogs。单击齿轮图标并选择 Upload WPILogs。`/home/lvuser/logs` 或 `/u/logs` 中的日志会自动下载到 `C:\Users\Public\Documents\FRC\Log Files\WPILogs`。

Custom Processing of Data Logs / 自定义处理数据日志

For more advanced processing of data logs (e.g. for processing of binary values that can't be converted to CSV), WPILib provides a **DataLogReader** class for reading data logs in [Java](#). The [data log format](#) is also documented.

对于数据日志的更高级处理（例如处理无法转换为 CSV 的二进制值），WPILib 提供 [Java DataLogReader](#) 类来读取数据日志。[数据日志格式](#)也有文档说明。

DataLogReader provides a low-level view of a data log, in that it supports iterating over a data log's control and data records and decoding of common data types, but does not provide any higher level abstractions such as a NetworkTables-like map of entries. The [printlog](#) example in [Java](#) demonstrates basic usage.

DataLogReader

提供数据日志的底层视图，支持遍历日志的控制记录和数据记录，并解码常见数据类型，但不提供类似 NetworkTables 条目映射的高级抽象。[Java 的 printlog 示例](#)演示基本用法。

Writing Your Own Sendable Classes / 编写自己的 Sendable 类

Since the **Sendable** interface only has one method, writing your own classes that implement **Sendable** (and thus automatically log values to and/or consume values from the dashboard) is extremely easy: just provide an implementation for the overridable **initSendable** method, in which setters and getters for your class' s fields are declaratively bound to key values (their display names on the dashboard).

由于 **Sendable** 接口只有一个方法，编写实现 **Sendable** 的自定义类（从而自动将值记录到仪表板，和/或从仪表板读取值）非常简单：只需实现可重写的 **initSendable** 方法，在其中以声明式方式把类字段的 setter 和 getter 绑定到键值（即它们在仪表板上的显示名称）。

For example, here is the implementation of **initSendable** from WPILib' s **BangBangController**:

例如，下面是 WPILib 的 **BangBangController** 中 **initSendable** 的实现：

```
@Override
public void initSendable(SendableBuilder builder) {
    builder.setSmartDashboardType("BangBangController");
    builder.addDoubleProperty("tolerance", this::getTolerance, this::setTolerance);
    builder.addDoubleProperty("setpoint", this::getSetpoint, this::setSetpoint);
    builder.addDoubleProperty("measurement", this::getMeasurement, null);
    builder.addDoubleProperty("error", this::getError, null);
    builder.addBooleanProperty("atSetpoint", this::atSetpoint, null);
}
```

To enable the automatic updating of values by WPILib “in the background” , **Sendable** data names are bound to getter and setter functions rather than specific data values. If a field that you wish to log has no defined setters and getters, they can be defined inline with a lambda expression.

为了让 WPILib “在后台” 自动更新数值，**Sendable** 数据名绑定的是 getter 和 setter 函数，而不是某个固定的数据值。如果要记录的字段没有已定义的 setter 和 getter，可以使用 lambda 表达式在行内定义。

The SendableBuilder Class / SendableBuilder 类

As seen above, the **initSendable** method takes a single parameter, **builder**, of type **SendableBuilder** (Java). This builder exposes methods that allow binding of getters and setters to dashboard names, as well as methods for safely ensuring that values consumed from the dashboard do not cause unsafe robot behavior.

如上所示，**initSendable** 方法接受一个参数 **builder**，其类型为 **SendableBuilder**。这个构建器提供将 getter 和 setter 绑定到仪表板名称的方法，也提供安全措施，确保从仪表板读取的数值不会造成不安全的机器人行为。

Databinding with addProperty Methods / 使用 addProperty 方法进行数据绑定

Like all WPILib dashboard code, **Sendable** fields are ultimately transmitted over **NetworkTables**, and thus the databinding methods provided by **SendableBuilder** match the supported **NetworkTables** data types:

与所有 WPILib 仪表板代码一样，**Sendable** 字段最终通过 **NetworkTables** 传输，因此 **SendableBuilder** 提供的数据绑定方法与 **NetworkTables** 支持的数据类型相对应：

- **boolean**: **addBooleanProperty**
- **boolean**: **addBooleanProperty**
- **boolean[]**: **addBooleanArrayProperty**
- **boolean[]**: **addBooleanArrayProperty**
- **double**: **addDoubleProperty**

- `double: addDoubleProperty`
- `double[]: addDoubleArrayProperty`
- `double[]: addDoubleArrayProperty`
- `string: addStringProperty`
- `string: addStringProperty`
- `string[]: addStringArrayProperty`
- `string[]: addStringArrayProperty`
- `byte[]: addRawProperty`
- `byte[]: addRawProperty`

Ensuring Safety with `setSafeState` and `setActuator` / 使用 `setSafeState` 和 `setActuator` 确保安全

Since **Sendable** allows users to consume arbitrary values from the dashboard, it is possible for users to pipe dashboard controls directly to robot actuations. This is extremely unsafe if not done with care; dashboards are not a particularly good interface for controlling robot movement, and users generally do not expect the robot to move in response to a change on the dashboard.

由于 **Sendable** 允许用户从仪表板读取任意数值，因此用户可能将仪表板控件直接接入机器人动作。如果处理不谨慎，这是极不安全的；仪表板并不是特别适合控制机器人运动的界面，而且用户通常也不会预期机器人会因仪表板上的变更而运动。

To help users ensure safety when interfacing with dashboard values, **SendableBuilder** exposes a `setSafeState` method, which is called to place any **Sendable** mechanism that actuates based on dashboard input into a safe state. Any potentially hazardous user-written **Sendable** implementation should call `setSafeState` with a suitable safe state implementation. For example, here is the implementation from the WPILib **PWMMotorController** class:

为帮助用户确保与仪表板数值交互时的安全，**SendableBuilder** 提供 `setSafeState` 方法；当 **Sendable** 机构根据仪表板输入执行动作时，可以调用该方法使其进入安全状态。任何可能存在危险、由用户编写的 **Sendable** 实现，都应以合适的安全状态实现来调用 `setSafeState`。例如，下面是 WPILib 的 **PWMMotorController** 类中的实现：

```
@Override
public void initSendable(SendableBuilder builder) {
    builder.setSmartDashboardType("Motor Controller");
    builder.setActuator(true);
    builder.setSafeState(this::disable);
    builder.addDoubleProperty("Value", this::get, this::set);
}
```

Additionally, users may call `builder.setActuator(true)` to mark any mechanism that might move as a result of **Sendable** input as an actuator. Currently, this is used by **Shuffleboard** to disable actuator widgets when not in **LiveWindow** mode.

此外，用户还可以调用 `builder.setActuator(true)`，将任何可能因 **Sendable** 输入而运动的机构标记为执行器。目前，**Shuffleboard** 用此标记在非 **LiveWindow** 模式下禁用执行器小组件。

Third-Party Telemetry Libraries / 第三方遥测库

Tip / 提示

Is your library not listed here when it should be? Open a pull request to add it!

如果你的库应列在这里却尚未列出，请提交 Pull Request 添加！

Several third-party logging utilities and frameworks exist that provide functionality beyond what is currently provided by WPILib:

现有若干第三方日志记录工具和框架，提供 WPILib 目前尚未提供的功能：

- [AdvantageKit](#) (Java only): “Log everything” -based logging framework with hooks for replaying logged data in [simulation](#).
- [AdvantageKit](#)（仅 Java）：一种基于“记录一切”理念的日志框架，提供在[仿真](#)中回放所记录数据的接口。
- [DSLOG](#): An alternate driver station log viewer.
- [DSLOG](#)：另一种 Driver Station 日志查看器。
- [DogLog](#) (Java only): A simple logging library for Java.
- [DogLog](#)（仅 Java）：一个适用于 Java 的简易日志记录库。